

DOSSIER

Raul Martin | Jorge Sanchez | Jorge Salvador

¡Visita el [GitHub del proyecto!](#)



ÍNDICE

- Descripción breve del proyecto
- Reparto de tareas
- Apartado Visual
- Base de Datos
 - Diseño
 - Conexión
- Sistemas y redes
- Programación
- Tests
- Integración y Despliegue Continuos
- Conclusión

Descripción breve del proyecto

En resumen, hemos desarrollado una aplicación en la que puedes jugar a tres juegos: el **tres en raya**, el **piedra, papel, tijera** y, su versión más compleja, el **piedra, papel, tijera lagarto, Spock**. Además, conforme ganes partidas **irás acumulando puntos** con los que **podrás canjear modificaciones estéticas** para diferentes pantallas de la misma.

Realmente esto no ha sido más que una excusa para poner en práctica todas las competencias que hemos ido aprendiendo a lo largo del curso en prácticamente todas sus asignaturas, más adelante especificaremos las tecnologías y herramientas utilizadas para cada apartado, pero el resultado final es bastante vistoso.

Para utilizar la aplicación

Es tan sencillo como **acceder a [este enlace](#)** al apartado de releases del GitHub del proyecto, **descargarse el instalable** de la aplicación adecuado a tu sistema operativo (.exe para Windows y .deb para Linux), **instalarla en tu equipo y lanzarla desde el acceso directo** que se generará en tu escritorio o tu menú de inicio. (En Linux se añadirá por defecto a la categoría “Games”).

¡Buena suerte en vuestras partidas y a conseguir muchos puntos!

Reparto de tareas

Se pueden observar varios cambios con respecto al reparto inicial en la fase idear:

- **Apartado Visual** - Jorge Salvador
- **Base de Datos** - Raul Martin
- **Conexión BD y programa** - Jorge Salvador
- **Sistemas y redes** - Raul Martin y Jorge Salvador
- **Programación** - Jorge Sanchez
- **Tests** - Jorge Sanchez
- **Integración y Despliegue** - Jorge Sanchez

Apartado Visual - Jorge Salvador

Base de Datos - Raul Martin

Diseño - Raul Martin

Conexión - Jorge Salvador

Sistemas y redes - Raul Martin y Jorge Salvador

Programación - Jorge Sánchez

A pesar de que en el curso solo hemos hecho proyectos en Java with ant, investigué que con maven la gestión de dependencias y la compatibilidad con JUnit, nuestra librería para test, era bastante mejor y por eso el paquete LogicaJuegos de nuestro programa está desarrollado en este entorno. Además decidimos usar el JDK 22 en vez del 1.8 que venimos usando durante el curso ya que nos proporcionaba opciones interesantes, como por ejemplo los rule switch.

La **estructura** de éste paquete principal de lógica es la siguiente:

Tenemos en el paquete como tal **dos clases abstractas**, Juego.java y Jugador.java, **que definen de manera robusta la arquitectura** del resto de nuestras clases juego y jugador; y estas dos clases además están completamente documentadas con Javadoc, tags incluidas.

En Juego hice uso de **herramientas** completamente **ajenas al contenido del grado y muy potentes** como los [genéricos](#), fundamentales para nuestra estructura de herencia entre juegos; y los [opcionales](#), que nos permiten manejar la posibilidad de que haya un dato que pueda no existir sin gestionar [NullPointerExceptions](#) directamente y de manera más robusta.

En relación a los recién mencionados opcionales, en el paquete principal tenemos también dos clases que implementan la interfaz [Supplier](#), también nueva para nosotros, que nos permiten definir qué excepciones y con qué mensaje se lanzarán en un caso u otro de uso de los opcionales, al intentar obtener el resultado de la partida o el ganador de la misma cuando no existen.

Luego tenemos el **sub-paquete TresEnRaya** que contiene todas las clases relacionadas con éste juego en concreto:

Partida3EnRaya, que hereda de Juego e implementa la lógica y bucle principal del juego;

Jugador3EnRaya, que hereda de Jugador e implementa un polimorfismo de varios de sus métodos específico para entenderse con la clase que gestiona la partida;

Jugada3EnRaya, que definimos como un turno, una posición del tablero y el jugador que la realiza;

EstadoTablero, que nos permite manejar tableros y sus posiciones de manera legible en código e intuitiva y comprobar cosas como si existe una amenaza¹ o si se ha logrado un tres en raya;

y Bot3EnRaya, que hereda de Jugador3EnRaya y es capaz de utilizar la información del tablero para tapar amenazas pero con cierta probabilidad de equivocarse, para que sea más emocionante.

El desarrollo de este minijuego fue bastante fluido ya que no requiere tener cosas sucediendo a la par, no era necesario gestionar varios hilos en su ejecución.

¹ Cuando un jugador tiene dos marcas alineadas de forma que, si el otro no tapa la última casilla vacía de esa "raya", ganará en su siguiente turno.

Con el piedra papel tijera sin embargo fué otra historia, ya que teníamos que gestionar que dos jugadores eligiesen gesto a la vez, en un tiempo límite:

La lógica para éste juego y su “primo malvado”, el piedra papel tijera lagarto Spock, se encuentra toda en el **sub-paquete PPT**.

Para el primero las clases son las siguientes:

PartidaPPT, que gestiona el bucle principal y los dos hilos necesarios para pedirle una jugada a la par a ambos jugadores, que se resuelve con el uso de **futuros completables** que se envían a los respectivos jugadores para que posteriormente y en un tiempo de menos de 10 segundos los complimenten con una jugada;
JugadorPPT, que es capaz de complimentar correctamente esta jugada;
JugadaPPT, que definimos como un turno, un gesto y el jugador que la realiza y además implementa la interfaz **Comparable** para definir qué gestos ganan contra cuales y nos permite hacer referencia a gestos de manera legible e intuitiva;
y BotPPT, que hereda de JugadorPPT y hace jugadas completamente aleatorias.

Para el segundo la estructura de clases es exactamente la misma solo que en vez de terminar en PPT terminan en PPTLS y todas heredan de sus equivalentes en PPT excepto por BotPPTLS, que hereda de JugadorPPTLS.

Además para **conectar éste paquete con el apartado visual**, que se encontraba en otro proyecto, le añadimos la lógica como dependencia y **desarrollé seis clases en el paquete pantallas.juegos**, hermanadas a las clases Partida y Jugador de cada uno de los juegos, que relacionan la lógica ya programada con los elementos y botones en sus respectivas pantallas. También modifiqué en la medida de lo necesario las mencionadas pantallas, añadiendo botones y eventos, paneles, labels y dialogs para diferentes funciones.

Tests - Jorge Sanchez

Los test son fundamentales para asegurar que el programa se comporta de la manera esperada y, como he mencionado anteriormente, los hemos realizado con la librería [JUnit](#), en concreto [JUnit 5](#), acompañada de otra librería llamada [Mockito](#), que nos permite utilizar mocks².

Nuestros tests, para ser honestos, podrían ser más completos pero los que tenemos son muy robustos y me han servido para aprender más de un par de cosas interesantes en relación a los mocks y el testing en general en Java.

Tenemos clases de test solo en el proyecto de lógica y siguiendo una estructura de paquetes completamente paralela a la del código fuente del proyecto, con clases como:

[EstadoTableroTest](#), que comprueba que ésta clase bastante compleja gestione de manera correcta las diferentes jugadas y posiciones posibles y encuentre patrones de amenaza y tres en raya de manera correcta;

[Jugada3EnRayaTest](#), que asegura que no se pueden crear jugadas de manera incorrecta o con datos erróneos y hace uso de tests parametrizados;

o [Partida3EnRayaTest](#), que depende mucho del uso de los mencionados mocks para comprobar que todo funciona correctamente indiferentemente de que pudiera haber errores, por ejemplo, en Jugador3EnRaya.

Por último mencionar que los tests se lanzan automáticamente al compilar el programa, de manera que no podrás hacer un build sin darte cuenta de si alguno falla.

² Herramienta que permite simular llamadas a otras clases definiendo respuestas específicas, sin tener que utilizar la otra clase como tal, de manera que puedes probar solo el funcionamiento de la clase que te interese sin depender de que el resto de clases que usa funcionen como se espera.

Integración y Despliegue - Jorge Sanchez

Integración continua

Conseguida mediante el uso de [git](#) y [Github](#) y el uso de muchas de sus funcionalidades más útiles, como por ejemplo:

- Ramas: hemos repartido el trabajo en **tres ramas, una por integrante del grupo**. Idealmente habríamos utilizado un flujo de trabajo más profesional con ramas específicas para cada feature o hotfix pero dado que más de la mitad del grupo tenían poca costumbre usando git me pareció excesivo.
- Pull requests: **nunca merges al main directamente!** Intenté inculcar el uso de esta herramienta para hacer revisiones de cambios por parte de varios integrantes del grupo y evitar que se subiera una versión defectuosa o incorrecta del proyecto a la rama principal.
- Issues (Github): para gestionar qué trabajo había que hacer, qué tareas bloqueaban otras, milestones, etc.
- Projects (Github): para mantener flujos de trabajo propios de metodologías ágiles, junto con los issues y los milestones. Además nos sirve también de diario de trabajo, tablero kanban y diagrama de gantt.

Me hice personalmente responsable de la integridad del proyecto en todos los puntos y fui el encargado de solucionar merges con conflictos.

Además, aunque de esto hablaré más en el apartado de despliegue, nos aseguramos de que todo el código es correcto ejecutando los tests propios automáticamente siempre que compilamos para producción.

Despliegue continuo

Realizado mediante **GitHub Actions** automáticamente cada vez que subimos una tag al repositorio remoto.

Es decir, cuando le ponemos nombre a una versión y subimos esa versión al GitHub **instantáneamente se lanza un proceso** que hace lo siguiente:

- Crea un entorno en la última versión de ubuntu, asegurándonos de usar el JDK correcto.
- Se clona la rama main del repositorio remoto.
- Testea y compila en caso de pasar las pruebas todo el código del proyecto de lógica.
- Se asegura de colocar los JARs de las dependencias en una carpeta para la compilación del proyecto del apartado visual, que es el que contiene el main: mueve el JAR compilado de la lógica y se baja de los repositorios de maven las demás dependencias.
- Testea y compila el proyecto del visual pasándole las dependencias y definiendo qué nombre tendrá nuestro JAR resultante en una carpeta, dist/.
- Mueve los JAR de las dependencias a una subcarpeta de la carpeta resultante, dist/lib/.
- Comprime todo en un ZIP y crea un draft de release en GitHub con él.
- **Entonces se lanzan dos tareas más en paralelo**, una en ubuntu nuevamente y otra en windows, que se ocupan de crear un instalable, .deb y .exe respectivamente, con la app compilada en la carpeta dist/ y unos iconos bien majos que hemos preparado. Estos instalables además crean accesos directos en ambos sistemas operativos.
- Por último, ambos instalables se añaden al draft de release anteriormente creado.

Como podéis ver está muy bien pensado, sólo nos hace falta añadirle al draft una descripción con el changelog y esas cosillas y tenemos un release hecho y derecho y **completamente standalone**, no requiere que el usuario tenga el JDK instalado porque incluye una versión reducida del mismo en el propio ejecutable.

Conclusión

Éste desafío ha sido una experiencia de aprendizaje muy enriquecedora y que nos ha permitido no solamente hacer uso de los conocimientos adquiridos durante el curso sino también expandirlos por nuestra cuenta y nos ha puesto en una situación de desarrollo mucho más cercana al mundo real que cualquier otra de las actividades hasta ahora.

Además éste aprendizaje ahora proviene de haber intentado, y habernos equivocado mucho en el proceso, sacar adelante un proyecto que, aunque no es ni mucho menos perfecto, en mi humilde opinión ha quedado bastante majo y esperamos que al menos os dé para un par de partiditas tontas al tres en raya o, quizás, que alguno se empeñe en desbloquear la tienda de puntos al completo!